

REVIO Project Setup Guide

1. Clone Repository

git clone <https://github.com/Tejashree2004/Review.git>

Move into the project:

```
cd Review
```

2. Database Setup

- Install PostgreSQL
- Create database:

```
CREATE DATABASE ReviewDB;
```

- Open PostgreSQL Query Tool / pgAdmin.
- Configure the database connection in:

```
backend/Review.API/appsettings.json
```

Set your PostgreSQL username and password:

```
"ConnectionStrings": {  
  "DefaultConnection":  
    "Host=localhost;Port=5432;Database=ReviewDB;Username=postgres;Password=YOUR_PASSWORD"  
}
```

Run Entity Framework Core migrations:

```
cd backend/Review.API
```

```
dotnet ef database update
```

3. Backend Setup

Go to the backend folder:

```
cd backend/Review.API
```

Restore dependencies:

```
dotnet restore
```

Build the project:

```
dotnet build
```

Run:

```
dotnet run
```

Backend:

<http://localhost:5213>

Swagger:

<http://localhost:5213/swagger>

4. Frontend Setup

Open a new terminal.

Go to:

`cd frontend`

Install dependencies:

`npm install`

Run the project:

`npm run dev`

Frontend:

<http://localhost:5173>

5. Frontend API Configuration

REVIO frontend communicates with the backend using:

<http://localhost:5213/api>

The API base URL is configured in the frontend API/service files.

Example:

```
const API = axios.create({  
  baseURL: "http://localhost:5213/api",  
});
```

If the backend port changes, update the API base URL accordingly.

6. Environment Variables

REVIO uses SMTP credentials for email/OTP functionality.

Configure the required email credentials in the local environment.

Example:

`SMTP_EMAIL=your-email@gmail.com`

SMTP_PASSWORD=your-app-password

For Gmail, use a Gmail App Password.

Do not commit passwords, SMTP credentials, or other secrets to GitHub.

Features Included

- User Signup / Login
- Email OTP Verification
- JWT Authentication
- Role-Based Authorization
- Reviewer / User Role
- Business Owner Role
- Admin Role
- Business Discovery
- Category-Based Business Browsing
- Search
- Top-Rated Businesses
- Business Details
- Create Business
- Edit Business
- Business Profile
- Business Photos
- Write Reviews
- Review Media Upload
- View Reviews
- My Reviews
- Favorites
- User Profile
- Edit Profile
- Notifications
- Owner Dashboard
- Admin Dashboard

- Owner Public Profile
 - Review Management
 - PostgreSQL Database
-

Important Files To Configure

Backend

backend/Review.API/appsettings.json

Used for:

- PostgreSQL connection string
- Application configuration

Program.cs

backend/Review.API/Program.cs

Used for:

- JWT configuration
 - Authentication / Authorization
 - CORS
 - Dependency Injection
 - API configuration
-

Frontend API

Frontend API/service files inside:

frontend/src/

Used for:

- Backend API URL
 - API requests
 - Authentication token handling
-

Database

backend/Review.API/Migrations/

Contains Entity Framework Core database migrations.

Common Errors

CORS Error

Check CORS configuration in:

backend/Review.API/Program.cs

Make sure the frontend URL is allowed:

http://localhost:5173

Database Connection Error

Verify:

- PostgreSQL is running.
- Database name is correct.
- PostgreSQL username is correct.
- PostgreSQL password is correct.
- PostgreSQL port is 5432.

Then run:

dotnet ef database update

npm install Issue

Delete:

node_modules

package-lock.json

Then run:

npm install

Frontend Cannot Connect to Backend

Verify backend is running:

http://localhost:5213

Check frontend API URL:

http://localhost:5213/api

OTP / Email Issue

Verify:

- Gmail email
- Gmail App Password
- SMTP configuration
- Internet connection

MailKit SMTP Configuration – .NET 10

1. Package Installation

REVIO uses MailKit for sending emails.

NuGet package:

Install-Package MailKit

2. Required Namespaces

using MailKit.Net.Smtp;

using MailKit.Security;

using MimeKit;

3. Email Service Implementation

REVIO contains an email service responsible for sending email messages.

The service uses:

- MimeMessage to create email messages
- MailKit.Net.Smtp.SmtpClient to connect to the SMTP server
- SendAsync() to send emails
- SMTP authentication for Gmail

4. SMTP Configuration

Gmail SMTP configuration:

Host: smtp.gmail.com

Port: 587

Security: StartTls

Email credentials should be supplied through the local environment/configuration.

5. Environment Variables

Sensitive SMTP credentials should be stored locally:

```
SMTP_EMAIL=your-email@gmail.com
```

```
SMTP_PASSWORD=your-app-password
```

The application reads the credentials from the environment/configuration used by the email service.

6. Secure SMTP Connection

MailKit uses StartTLS for encrypted SMTP communication:

```
await client.ConnectAsync(  
    host,  
    port,  
    SecureSocketOptions.StartTls  
);
```

7. SMTP Authentication

The SMTP client authenticates using the configured email and password:

```
await client.AuthenticateAsync(  
    email,  
    password  
);
```

8. Sending Email

Email is sent using:

```
await client.SendAsync(message);
```

After sending, the SMTP connection is closed:

```
await client.DisconnectAsync(true);
```

9. HTML Email Support

REVIO can use HTML email content through:

```
new TextPart("html")
```

This can be used for formatted OTP and other email messages.

10. Dependency Injection

The email service is registered through ASP.NET Core Dependency Injection in:

Program.cs

Example:

```
builder.Services.AddScoped<EmailService>();
```

11. Error Handling

Email operations should be handled using exception handling:

```
try
{
    // Send email
}
catch (Exception ex)
{
    // Handle email error
}
```

This prevents email failures from crashing the application unexpectedly.

Technologies Used

Frontend

- React.js
- Vite
- React Router DOM
- Axios
- React Icons
- Responsive CSS

Backend

- ASP.NET Core Web API
- .NET 10

- Entity Framework Core
- PostgreSQL
- JWT Authentication
- BCrypt
- MailKit SMTP
- Swagger